

Copy everything below and paste it into Word, Google Docs, Notion, or any document editor.

# Caliper Studio: Image Upscale and Vectorize App

## Conversation Answers, Product Plan, and Technical Design

Topic: App plan and design for upscaling images and vectorizing raster images into SVG/vector outputs

Working product name: Caliper Studio

Purpose: Build a web, mobile, or AI bot product that takes a user image as input and outputs an upscaled and/or vectorized version.

Core outputs: SVG, PNG, JPEG, WebP, PDF, EPS, DXF, and ZIP bundles

Recommended launch surface: Web app + API first, AI bot second, mobile app after the core quality is stable

Date: May 5, 2026

---

## 1. Research Findings and Product Direction

I reviewed the attached Caliper PRD and treated it as the core engine foundation. I also researched the five example products you shared:

1. Vectorizer.io
2. Vectorizer.AI
3. Topaz Labs
4. Poe TopazLabs bot
5. AI Logo Maker mobile app on Google Play

The main conclusion is that the best product should not be only an image upscaler or only a raster-to-SVG converter. The stronger product is a guided image-processing workflow that decides whether to upscale, vectorize, or do both.

The product should accept a raster image from the user and return:

- A higher-quality raster image.
- A clean editable vector image.
- Or both, depending on the user's image and goal.

The existing Caliper PRD is already a strong vectorization foundation. The new product should wrap Caliper with a user-facing app, API, AI bot, preview workflow, billing, and job orchestration.

---

## 2. What I Found from the Five Example Products

### 2.1 Vectorizer.io

Vectorizer.io is closest to a traditional web vectorizer. It converts raster images such as PNG, BMP, and JPEG into vector formats such as SVG, EPS, and DXF.

It describes vectorization as converting pixel color information into simple geometric objects. This usually means detecting areas of similar brightness or color and expressing them as lines, circles, curves, and filled shapes.

Vectorizer.io also exposes an experimental “Enhance image with AI” option. This feature improves an image before vectorization by removing noise, simplifying colors, and improving edges.

Its API shows the kinds of settings that a serious vectorizer needs:

- Model type.
- Tracing algorithm.
- Detail level.
- Antialiasing settings.
- Minimum shape area.
- Color merging.
- Transparency handling.
- Background removal.
- Roundness.
- Palette import.
- Filled-shape tracing.
- Stroked or centerline tracing.

A major opportunity is that Vectorizer.io’s API page says API access is no longer available to new users. That creates space for a developer-friendly vectorization API.

### *Design lessons from Vectorizer.io*

The product should include automatic mode for casual users, but it should also expose advanced controls for professional users.

Important features include:

- Background removal.
- Palette control.
- Minimum shape area.
- Color merging.
- Centerline tracing.
- Free preview before paid download.

---

## 2.2 Vectorizer.AI

Vectorizer.AI is the strongest reference for a premium vectorization product.

It positions itself as a fully automatic AI bitmap-to-vector service for PNG, JPG, GIF, BMP, and WebP inputs. It exports SVG, PDF, EPS, DXF, and PNG.

Vectorizer.AI claims to combine deep learning networks with classical algorithms. It also describes a proprietary computational geometry system called a “Vector Graph.”

Important capabilities include:

- Whole-shape fitting.
- Clean corners.
- Curve support.
- Symmetry modelling.
- Adaptive simplification.
- Palette control.
- Sub-pixel precision.
- Full alpha/transparency support.
- CPU/GPU acceleration.
- Preview before download.
- API access.
- Re-download in multiple output formats.

### *Design lessons from Vectorizer.AI*

The app should not only generate raw SVG paths. It should try to preserve meaningful geometry where possible, such as:

- Circles.
- Ellipses.
- Rounded rectangles.
- Stars.
- Lines.
- Arcs.
- Bezier curves.

This matters because professional users want SVG files that are editable, clean, and compatible with design tools.

Palette control, transparency support, sub-pixel placement, and adaptive simplification should be considered core quality features, not advanced extras.

---

## 2.3 Topaz Labs

Topaz Labs is the best reference for the image upscaling branch of the product.

Topaz is known for AI-powered photo and image enhancement. Topaz Gigapixel is used to upscale images, graphics, artwork, and photos while preserving detail and clarity.

Topaz Labs supports both local and cloud rendering. Its API uses an asynchronous job workflow with:

- Job submission.
- A process ID.
- Status polling.
- Download endpoints.
- ETA estimates.

Topaz model selection is also useful. It separates models or workflows for:

- Precision upscaling.
- Generative upscaling.
- Creative upscaling.
- Text and shapes.
- Low-resolution inputs.
- Transparent PNGs.
- Face recovery.
- Art and CGI.

### *Design lessons from Topaz Labs*

Upscaling should not use one model for every input. The system should route images based on content type.

Different images need different upscaling strategies:

- Photos need detail recovery.
- Logos need clean edges.
- Text needs shape preservation.
- Transparent PNGs need alpha preservation.
- Pixel art needs exact grid preservation.
- AI art needs artifact cleanup.

Large images also require tiled inference, output size limits, and clear cost estimation.

Output megapixels are a natural billing unit for raster enhancement.

---

## 2.4 Poe TopazLabs Bot

The Poe TopazLabs bot is a useful reference for packaging image upscaling inside a chat interface.

The bot accepts uploaded images and exposes simple parameters such as:

- `upscale`
- `output_height`

- `output_width`
- `generated`

The important lesson is that a bot should not expose every advanced control. It should provide a simple natural-language interface that calls the same backend API as the web app.

Example bot commands could be:

- “Upscale this 4x and keep transparency.”
- “Turn this logo into a clean SVG with max 8 colors.”
- “Make this suitable for vinyl cutting and remove the white background.”
- “Upscale 2x, then vectorize as PDF and SVG.”

The bot should return:

- A brief diagnosis.
- A preview image.
- Download links.
- Suggested alternate settings.

---

## 2.5 AI Logo Maker Mobile App

The Google Play app you linked is more of an AI logo generator than a raster-to-SVG vectorizer.

It appears to focus on:

- Prompt-to-logo generation.
- Logo design suggestions.
- Customization of colors, fonts, layouts, and styles.
- Exporting high-quality logo files.

This is still useful as a mobile UX reference. Mobile should be simple, preset-driven, and focused on quick logo cleanup or generation, not a full professional tracing interface.

### *Design lessons from the mobile app*

A mobile version should not mirror the full web app interface. It should focus on:

- Import from camera or gallery.
- One-tap cleanup.
- Simple presets.
- Quick preview.
- Simple palette edits.
- Easy sharing and export.

Logo generation could be a future adjacent feature, but it should not be part of the MVP.

---

### 3. Recommended Product Direction

Build this as:

1. Web app + API first.
2. AI bot second.
3. Mobile app third.

The reason is practical. Vectorization quality requires:

- Preview.
- Zoom.
- Palette controls.
- Background controls.
- Path simplification.
- Export options.
- Download bundles.
- Before/after comparison.

These are much easier to validate in a web app.

The AI bot can be a thin workflow layer over the same backend API. The mobile app should come after the presets and processing pipeline are reliable.

---

### 4. Working Product Name

The working product name is:

*Caliper Studio*

The core promise:

“Upload a low-quality raster image. Get back a sharper raster image, a clean editable SVG, or both.”

---

### 5. Relationship to the Existing Caliper PRD

Your existing Caliper PRD is already a serious vectorization engine specification.

It includes:

- Rust-native engine design.
- Adaptive Assayer router.

- CPU/GPU/NPU-aware architecture.
- SVG/PDF/EPS/DXF output.
- Seven-stage tracing pipeline.
- Professional vector output goals.

The new product should not replace Caliper. It should wrap Caliper.

Recommended relationship:

Caliper Core

= Local/vectorization engine.

Caliper Studio

= Web app + API + AI bot + later mobile client.

Upscale Engine

= Separate GPU/AI service that runs before Caliper when useful.

The important product leap is the Assayer. In the app, the Assayer becomes the system that tells the user what workflow is best.

Example:

Low-res logo detected.

Recommended workflow:

4x conservative upscale → 12-color vectorization → SVG + PDF export.

That is the difference between a simple converter and a professional tool.

---

## 6. Proposed App Modes

The MVP should have three main modes.

### 6.1 Upscale Only

Input:

- PNG.
- JPEG.
- WebP.
- BMP.
- GIF first frame.

Output:

- PNG.
- JPEG.
- WebP.

Scale options:

- 2x.

- 4x.
- Custom dimensions.

Use cases:

- Product photos.
  - Social media graphics.
  - AI-generated images.
  - Old low-resolution images.
  - Transparent PNG enhancement.
- 

## 6.2 Vectorize Only

Input:

- Clean logo.
- Icon.
- Line art.
- Drawing.
- Sticker design.
- Simple illustration.

Output:

- SVG.
- PDF.
- EPS.
- DXF.
- PNG preview.

Use cases:

- Logo conversion.
  - Sticker cutting.
  - Vinyl cutting.
  - Laser engraving.
  - Print preparation.
  - Design editing.
- 

## 6.3 Upscale + Vectorize

Input:

- Small logo.
- Compressed image.
- Low-resolution AI asset.
- Blurry screenshot.
- Old website logo.
- Small transparent PNG.

Output:

- Clean SVG.
- High-resolution PNG.
- Optional PDF/EPS/DXF bundle.

Best first commercial use case:

Low-resolution logo → 4x upscale → clean 8-16 color SVG → PNG/PDF bundle.

---

## 7. Suggested Processing Pipeline

The full processing pipeline should be:

User image

- Validate and decode
- Analyze with Assayer
- Optional crop/background/alpha handling
- Optional AI upscale
- Optional vectorization with Caliper
- Preview generation
- Export SVG/PDF/EPS/DXF/PNG
- Quality metrics
- Download or API webhook

For upscaling, the MVP can start with a third-party API to move fast. Later, the system can replace third-party upscaling with internal models.

Possible technical directions include:

- Real-ESRGAN-style practical restoration.
- SwinIR-style transformer restoration.
- Text/shape-preserving upscalers.
- Transparent PNG upscalers.
- Logo-specific edge-preserving upscalers.

For vectorization, useful references include:

- Potrace for bitmap tracing concepts.
- VTracer for color vectorization and Rust relevance.
- Caliper as the long-term professional engine.

---

## 8. Core Architecture

Recommended high-level architecture:

Client apps

- Web app
- Mobile app
- AI bot
- Public API clients

→ API Gateway / App Backend

- Authentication
- Billing
- Upload sessions
- Job creation
- Webhooks

→ Job Orchestrator

- Queue
- Scheduler
- Worker leases
- Retries
- Cancellation
- Progress events

→ Storage and Data Layer

- Object storage for input, previews, and outputs
- Postgres for users, jobs, assets, results, credits
- Redis for short-lived cache, rate limits, and progress

→ Processing Workers

- Assayer analyzer
  - GPU upscale worker
  - Caliper vectorization worker
  - SVG preview/rasterization worker
  - Export/ZIP worker
-

## 9. Recommended Technology Stack

### 9.1 Frontend

Recommended options:

- Next.js.
- Remix.
- React preview layer.
- SVG/canvas-based before/after preview.
- WebAssembly preview for light client-side inspection if needed.

Mobile after MVP:

- Flutter.
- React Native.

### 9.2 Backend

Recommended options:

- Rust Axum, because it aligns with Caliper.
- TypeScript/NestJS, if faster product iteration is preferred.

Core backend components:

- Postgres.
- Redis.
- Temporal, BullMQ, or similar job orchestration.
- S3-compatible object storage.
- GPU worker pool.
- CPU-heavy vectorization worker pool.
- Optional Kubernetes GPU node groups.

### 9.3 Processing

Vectorization:

- Caliper core as a Rust library or service.
- VTracer as a possible baseline or comparison point.
- Potrace-like logic for monochrome or centerline modes.

Upscaling:

- Third-party API for MVP.
- ONNX Runtime, TensorRT, or Triton Inference Server later.
- GPU workers for internal inference.
- Tiled processing for large images.

Preview/export:

- SVG renderer.
  - Resvg or similar rendering tool.
  - SVG0-like optimization, but avoid destroying editability.
  - ZIP packaging for multi-format downloads.
- 

## 10. MVP Feature List

The first version should include the following.

### 10.1 Upload

- Drag and drop.
- File picker.
- Paste image.
- URL import.

### 10.2 Input Formats

- PNG.
- JPG/JPEG.
- WebP.
- BMP.
- GIF first frame.

TIFF can be added soon after MVP.

### 10.3 Modes

- Upscale only.
- Vectorize only.
- Upscale + vectorize.

### 10.4 Presets

- Logo/Icon.
- Sticker/Vinyl/Cutter.
- Line Art/Sketch.
- Photo to Poster Vector.
- AI Art Cleanup.
- Transparent PNG Upscale.

## 10.5 Controls

- Upscale factor.
- Output width.
- Output height.
- Max colors.
- Remove background.
- Transparency threshold.
- Simplify level.
- Corner sharpness.
- Shape fitting on/off.
- Centerline mode.
- Preserve alpha.
- Output format.

## 10.6 Outputs

- SVG.
- PNG.
- JPEG.
- WebP.
- PDF.
- EPS.
- DXF.
- ZIP bundle.

## 10.7 Business Features

- Free preview.
  - Paid production download.
  - User account.
  - Job history.
  - Delete files.
  - Credit system.
  - API key.
  - Webhook.
-

## 11. Product Rule: Do Not Promise Perfect SVG for Every Image

The product should not promise that every image can become a perfect editable SVG.

The app should classify the image and guide expectations.

Recommended classifications:

Logo/icon:

Excellent SVG candidate.

Line art:

Excellent SVG or centerline candidate.

Pixel art:

Preserve exact pixels. Do not AI-smooth unless requested.

Photo:

Better as upscale or artistic poster vector. Not usually a clean editable SVG.

Screenshot/text:

Use text-shape-preserving upscale, then vectorize carefully.

Complex illustration:

Possible vector output, but may produce many paths.

This expectation-setting will reduce refunds and make the product feel smarter.

---

## 12. User Experience Design

### 12.1 Web App Flow

Step 1: Landing page

The landing page should include:

- Large drop zone: “Drop PNG/JPG/WebP to upscale or vectorize.”
- Three cards: Upscale, Vectorize, Upscale + Vectorize.
- Example gallery with before/after sliders.
- Clear promise: sharper raster output or clean editable vector output.

Step 2: Upload and analyze

The user uploads an image.

The backend validates:

- File type.
- File size.
- Dimensions.
- Alpha/transparency.
- Color count.
- Noise level.



The user can download:

- Individual selected formats.
- ZIP bundle.
- SVG + PNG + PDF bundle.
- Print/cutter-ready bundle.

The user should also be able to:

- Save the job as a project.
- Re-run with changed parameters.
- Delete the asset.
- Share result links.

---

## 12.2 Mobile App Flow

Mobile should be a thin client first.

Core flow:

- Import from camera.
- Import from gallery.
- Import from Files.
- Import from share sheet.
- Choose one of four simple cards:
  - Clean Logo.
  - Upscale Photo.
  - Make SVG.
  - Sticker/Cutter.
- Preview result.
- Apply small edits:
  - Crop.
  - Background.
  - Colors.
  - Simplify.
- Export/share.

Mobile should not expose every professional parameter. The mobile app's job is speed and clarity.

---

## 12.3 AI Bot Flow

The bot should call the same API as the web app.

Example commands:

- “Upscale this 4x and keep transparency.”
- “Turn this logo into a clean SVG with max 8 colors.”
- “Make this suitable for vinyl cutting. Remove the white background.”
- “Upscale 2x, then vectorize as PDF and SVG.”

Bot response should include:

- Brief diagnosis.
- Preview image.
- Download links.
- Suggested alternate settings.

Example bot response:

“Your image looks like a low-resolution logo with transparency. I recommend 4x conservative upscale followed by 10-color SVG vectorization. I created a preview. You can download SVG, PNG, and PDF below.”

---

## 13. Technical Architecture

### 13.1 High-Level Architecture

Client apps:

- Web app.
- Mobile app.
- AI bot.
- Public API clients.

Backend:

- API gateway.
- Authentication.
- Billing.
- Upload sessions.
- Job creation.
- Webhooks.

Job orchestration:

- Queue.

- Scheduler.
- Worker leases.
- Retry and cancellation.
- Progress events.

Storage:

- Object storage for inputs, previews, and outputs.
- Postgres for metadata.
- Redis for cache, rate limits, and progress.

Processing workers:

- Analyzer/Assayer worker.
- Upscale GPU worker.
- Vectorization worker using Caliper core.
- Preview rasterizer.
- Export/packaging worker.

---

## 13.2 Build vs Buy Decision

For fastest MVP:

- Use a third-party image upscaling API for the raster upscale branch.
- Use Caliper, VTracer, or a Caliper/VTracer hybrid for vectorization.

For long-term control:

- Train or fine-tune internal upscalers.
- Keep the vectorization core internal through Caliper.
- Reduce third-party cost of goods sold.
- Add offline or on-device options later.

---

## 14. Processing Pipeline Design

### 14.1 Unified Pipeline

Input image

- Validate and decode
- Assayer analysis
- Optional crop/background/alpha handling
- Optional upscale branch
- Optional vectorize branch
- Preview generation
- Export generation

- Quality metrics
- Download/result retention

---

## 14.2 Assayer Routing

The Assayer should produce a machine-readable plan.

Example:

```
{
  "image_class": "low_res_logo",
  "recommended_mode": "upscale_then_vectorize",
  "upscale": {
    "enabled": true,
    "factor": 4,
    "model_family": "logo_text_shapes",
    "preserve_alpha": true
  },
  "vectorize": {
    "enabled": true,
    "max_colors": 12,
    "path_mode": "spline",
    "shape_fitting": true,
    "symmetry": true,
    "output_formats": ["svg", "pdf"]
  },
  "rationale": [
    "short_edge_under_512_px",
    "low_color_count",
    "sharp_edges",
    "alpha_channel_present"
  ]
}
```

---

## 14.3 Upscaling Branch

Steps:

1. Normalize image orientation and color profile.
2. Detect image class:
  - Photo.
  - Face/portrait.
  - Logo/text.
  - Transparent asset.
  - Illustration.
  - AI art.
  - Pixel art.
3. Select model and scale:

- Photo: standard or high-fidelity model.
  - Low-resolution/compressed: low-resolution restoration model.
  - Logo/text/screenshot: edge-preserving text/shape model.
  - Illustration/CG/anime/AI art: art/CG model.
  - Transparent PNG: preserve alpha and avoid matte halos.
  - Pixel art: nearest-neighbor or exact grid mode unless user explicitly asks for AI smoothing.
4. Tile large images with overlap.
  5. Run inference on GPU.
  6. Blend seams and restore alpha.
  7. Optional denoise/sharpen/compression repair.
  8. Generate preview and output.

---

## 14.4 Vectorization Branch

Use the attached Caliper PRD as the technical core.

Steps:

1. Decode and preprocess.
2. Quantize colors in a perceptual color space.
3. Detect edges and connected components.
4. Trace contours with hole hierarchy.
5. Simplify contours.
6. Fit splines and geometric primitives.
7. Detect symmetry and enforce constraints where confidence is high.
8. Stitch tiles.
9. Assemble layers.
10. Export SVG/PDF/EPS/DXF.
11. Rasterize output for preview.
12. Compute visual difference metrics.

---

## 14.5 When to Upscale Before Vectorizing

Upscale first when:

- Input is a small logo or icon below about 512 px on the short side.
- Compression artifacts obscure edges.
- Anti-aliased pixels are too sparse for clean sub-pixel boundary estimation.

- The user wants smoother SVG curves from a low-resolution raster.
- The input is a low-resolution transparent PNG.

Do not upscale first when:

- The image is pixel art and the user wants exact blocky geometry.
  - The input is already high-resolution and clean.
  - The goal is centerline tracing of technical drawings.
  - The image is photographic and the user only wants a posterized artistic vector.
  - The upscaler would invent details that hurt vectorization.
- 

## 15. API Design

### 15.1 Core Endpoints

```
POST /v1/assets/upload-session
POST /v1/assets/{asset_id}/complete
POST /v1/jobs
GET /v1/jobs/{job_id}
GET /v1/jobs/{job_id}/events
POST /v1/jobs/{job_id}/cancel
GET /v1/results/{result_id}/download
DELETE /v1/assets/{asset_id}
POST /v1/webhooks/test
```

---

### 15.2 Create Job Request Example

```
{
  "asset_id": "asset_123",
  "mode": "upscale_then_vectorize",
  "preset": "logo_icon",
  "upscale": {
    "factor": 4,
    "output_width": null,
    "output_height": null,
    "preserve_alpha": true
  },
  "vectorize": {
    "max_colors": 12,
    "background": "auto_remove_white",
    "simplify": "medium",
    "corner_sharpness": "auto",
    "shape_fitting": true,
    "centerline": false
  },
  "outputs": ["svg", "png", "pdf"],
  "retention_days": 7,
  "webhook_url": "https://customer.example/webhooks/caliper"
}
```

---

## 15.3 Job Response Example

```
{
  "job_id": "job_123",
  "status": "queued",
  "eta_seconds": 28,
  "credits_estimated": 3,
  "assayer": {
    "image_class": "low_res_logo",
    "recommended_mode": "upscale_then_vectorize",
    "confidence": 0.91
  }
}
```

---

## 15.4 Webhook Event Example

```
{
  "event": "job.completed",
  "job_id": "job_123",
  "asset_id": "asset_123",
  "results": [
    {
      "result_id": "res_svg_123",
      "format": "svg",
      "download_url": "signed-url-or-api-download-url"
    },
    {
      "result_id": "res_png_123",
      "format": "png",
      "download_url": "signed-url-or-api-download-url"
    }
  ],
  "metrics": {
    "path_count": 184,
    "colors": 10,
    "svg_bytes": 52831,
    "visual_match_score": 0.96
  }
}
```

---

## 16. Data Model

Core tables:

### 16.1 Users

Stores:

- Account details.
- Role.

- Plan.
- Billing status.
- Created date.
- Last login.

## 16.2 Assets

Stores:

- Original file metadata.
- Dimensions.
- Checksum.
- Storage URI.
- Retention policy.
- File type.
- Upload source.

## 16.3 Jobs

Stores:

- Mode.
- Preset.
- Status.
- Credit estimate.
- Actual cost.
- Timestamps.
- User ID.
- Asset ID.

## 16.4 Job Stages

Stores:

- Stage name.
- Status.
- Started time.
- Ended time.
- Logs.
- Retry count.
- Error details.

## 16.5 Results

Stores:

- File format.
- Storage URI.
- Preview URI.
- Byte size.
- Dimensions.
- Metrics.
- Download count.

## 16.6 Presets

Stores:

- Preset name.
- Version.
- Default settings.
- Compatibility flags.

## 16.7 API Keys

Stores:

- Hashed keys.
- Scopes.
- Last used timestamp.
- Rate limits.

## 16.8 Credit Ledger

Stores:

- Credit grants.
- Credit debits.
- Refunds.
- Expiry date.
- Billing reference.

## 16.9 Webhooks

Stores:

- Target URL.
- Signing secret.
- Delivery attempts.

- Last status.
  - Failure logs.
- 

## 17. Quality Metrics

### 17.1 Raster Upscale Metrics

Track:

- Output dimensions.
- Scale factor.
- Estimated compression reduction.
- Artifact reduction.
- Runtime.
- GPU memory usage.
- Before/after preview score.
- Face/detail confidence if applicable.

### 17.2 Vector Output Metrics

Track:

- Rasterized SVG vs input visual match score.
- Color delta summary.
- Path count.
- Primitive count.
- File size.
- Minimum shape area removed.
- Transparency preservation score.
- Compatibility warnings.
- Number of colors.
- Shape fitting confidence.

### 17.3 User-Facing Quality Badges

Possible badges:

- Print ready.
- Cutter ready.
- Editable SVG.
- Transparent background preserved.

- Low path count.
  - Too photographic for clean SVG.
  - Low confidence – try more colors.
  - Large SVG – increase simplification.
  - Good candidate for vectorization.
- 

## 18. Security and Privacy

Must-have controls:

- Validate magic bytes, not just file extension.
  - Strip or optionally preserve EXIF metadata.
  - Enforce max dimensions.
  - Enforce max file size.
  - Detect decompression bombs.
  - Decode images in sandboxed workers.
  - Use signed upload URLs.
  - Use signed download URLs with short TTLs.
  - Encrypt objects at rest.
  - Allow user-controlled deletion.
  - Default retention:
    - 24 hours for anonymous users.
    - 7 days for logged-in users.
    - Configurable for API users.
  - Do not train on customer images unless explicitly opted in.
  - Block SSRF for URL imports.
  - Prevent DNS rebinding attacks.
  - Block private IP URL imports.
  - Rate-limit by account, IP, API key, and file hash.
  - Add webhook signatures.
  - Keep logs free of sensitive image contents where possible.
- 

## 19. Licensing and Legal Risk

The attached Caliper PRD identifies GPL-3.0-or-later licensing.

If Caliper core is distributed inside desktop or mobile apps, GPL source obligations must be respected.

A hosted SaaS wrapper has different obligations than distributing binaries, but the architecture should keep licensing boundaries explicit from the beginning.

Recommended actions:

- Audit every engine license.
  - Audit every model license.
  - Audit every dependency license.
  - Separate GPL-related decisions from commercial app distribution.
  - Avoid embedding GPL code into closed-source distributed binaries unless the business is prepared to comply.
  - Consider a dual-license strategy if appropriate.
  - Document source distribution obligations from day one.
- 

## 20. Billing Design

Suggested credit model:

- Free tier: limited previews, watermarked output, or low-resolution SVG preview.
- Upscale: based on output megapixels.
- Vectorize: based on input megapixels and complexity.
- Additional formats: small extra fee if generated from retained result.
- Batch/API: volume discounts.
- Retention beyond default: storage fee.

Example credit formula:

`upscale_credits = ceil(output_megapixels / 24)` for standard upscale

`vector_credits = base 1 + complexity_multiplier + high_resolution_multiplier`

`bundle_download = 0.1 to 0.25 credits per extra format`

---

### 20.1 Suggested Plans

#### *Free*

- Limited previews.
- Starter credits.
- Watermarked or low-resolution output.
- Small file size limit.

### *Creator*

- Monthly credits.
- No watermark.
- Web UI.
- Standard output sizes.

### *Pro*

- Batch processing.
- Advanced controls.
- Larger files.
- More output formats.
- Job history.

### *API*

- API keys.
- Webhooks.
- Higher rate limits.
- Batch jobs.
- Usage dashboard.

### *Enterprise*

- Private deployment.
  - Custom retention.
  - SSO.
  - SLA.
  - Dedicated support.
  - Optional local/offline processing.
- 

## 21. Roadmap

### *Phase 0: Discovery and Prototype — 2 Weeks*

Tasks:

- Build a test set of 100-200 sample images.
- Include logos, icons, photos, AI images, screenshots, line art, bad compressed inputs, transparent PNGs, and pixel art.
- Run samples through Vectorizer.io, Vectorizer.AI, Topaz, and the prototype.
- Define a quality rubric.

- Build a proof of concept:
    - Upload.
    - Upscale.
    - Vectorize.
    - SVG preview.
  - Decide temporary upscaler provider vs internal model.
- 

## *Phase 1: MVP Web App — 6 to 8 Weeks*

Tasks:

- Build authentication.
- Build upload flow.
- Add object storage.
- Add job queue.
- Add Caliper/VTracer-based vectorization worker.
- Add third-party or internal upscaling worker.
- Add web preview.
- Add download flow.
- Add basic billing/credits.
- Add deletion and retention controls.

MVP result:

A user can upload a low-resolution logo, preview the improved output, and download SVG/PNG/PDF results.

---

## *Phase 2: Professional Controls and API — 6 to 8 Weeks*

Tasks:

- Add public API v1.
- Add webhooks.
- Add batch processing.
- Add preset tuning.
- Add palette editor.
- Add background removal controls.
- Add SVG optimizer.
- Add compatibility profiles.
- Add cutter/print/export presets.

---

## *Phase 3: AI Bot and Mobile Thin Client — 4 to 6 Weeks*

Tasks:

- Add bot commands over existing API.
- Add mobile import/share/export flow.
- Add push notifications for completed jobs.
- Add simple mobile presets.
- Add one-tap logo cleanup.

---

## *Phase 4: Proprietary Quality Moat — Ongoing*

Tasks:

- Train or fine-tune upscaler models for:
  - Logos.
  - Text.
  - Transparent images.
  - AI art.
  - Photos.
- Improve primitive fitting.
- Improve symmetry detection.
- Add optional OCR/text replacement layer.
- Add local/offline desktop mode.
- Add WebAssembly mode for lightweight client-side jobs.
- Add enterprise privacy features.

---

## **22. MVP Success Metrics**

### **22.1 Activation**

Targets:

- 60% of new users upload an image.
- 40% generate a preview.
- 15% download a result.

### **22.2 Quality**

Targets:

- 80% of logo/icon jobs receive a thumbs-up or no re-run.

- Median SVG path count below a defined threshold for the golden logo test set.
- Median preview time under 15 seconds for standard images.
- Median production time under 60 seconds for common images.

## 22.3 Business

Targets:

- Preview-to-paid conversion above 8%.
- API customer retention above 60% after the first month.
- Gross margin above 70% after replacing third-party upscaling with internal models.

## 22.4 Reliability

Targets:

- 99.5% job completion for supported inputs.
- Under 1% corrupted or invalid output reports.
- No unauthorized data retention beyond configured policy.

---

## 23. Key Risks and Mitigations

### *Risk 1: Users Expect Perfect SVG from Photos*

Mitigation:

Detect photos and present “artistic poster vector” language. Do not promise clean editable logo-style SVG from complex photos.

---

### *Risk 2: Upscaling Invents Details That Harm Vectorization*

Mitigation:

Use conservative logo/text models before vectorization. Let users compare:

- Original → vectorized.
- Upscaled → vectorized.

---

### *Risk 3: SVG Files Become Huge*

Mitigation:

Use:

- Path-count budgets.
- Simplification.
- Palette reduction.

- Primitive fitting.
  - Warnings.
  - Compatibility profiles.
- 

### *Risk 4: GPU Costs Are Too High*

Mitigation:

- Price by output megapixels.
  - Tile efficiently.
  - Cache identical file hashes.
  - Use third-party API only during MVP.
  - Move to internal inference later.
- 

### *Risk 5: Legal or Licensing Issues*

Mitigation:

- Audit every engine.
  - Audit every model.
  - Separate GPL code decisions from commercial distribution.
  - Document source obligations.
  - Consider dual licensing if needed.
- 

### *Risk 6: Malicious Images or Decompression Bombs*

Mitigation:

- Strict validation.
  - Sandboxed decoding.
  - Max pixels.
  - Timeouts.
  - Memory quotas.
  - Isolated workers.
- 

## **24. Recommended First Build**

Build Caliper Studio as a web app and API first, not as mobile-first.

The web app proves:

- Quality.

- Preview UX.
- Billing.
- Developer workflows.
- Export compatibility.
- Preset reliability.

Add the AI bot quickly because it is only another API client.

Build the mobile app after the pipeline quality and presets are reliable.

The first production-grade workflow should be:

Upload low-res logo

- Assayer recommends upscale + vectorize
- 4x conservative upscale preserving alpha
- Color quantization with max 8-16 colors
- Contour trace and primitive fitting
- Interactive SVG preview
- Download SVG + PNG + PDF bundle

This is the clearest commercial use case and the best fit for Caliper's strengths.

---

## 25. Suggested MVP Screens

### *Screen 1: Landing Page*

Elements:

- Hero headline.
- Upload drop zone.
- Three mode cards.
- Before/after examples.
- Pricing preview.
- Trust badges.

Suggested headline:

"Turn low-resolution images into sharp PNGs and clean editable SVGs."

---

### *Screen 2: Upload and Analyze*

Elements:

- Image thumbnail.
- File details.
- Detected type.
- Recommended workflow.

- Confidence score.
- Suggested preset.

Example:

Detected: Low-resolution logo with transparent background.

Recommended: Upscale 4x, then vectorize as 12-color SVG.

---

### *Screen 3: Settings*

Simple controls:

- Mode.
- Preset.
- Output formats.
- Upscale factor.
- Max colors.
- Background removal.
- Simplification.

Advanced drawer:

- Color palette.
- Minimum shape size.
- Corner sharpness.
- Curve tolerance.
- Centerline mode.
- Alpha threshold.
- Stroke/fill preference.

---

### *Screen 4: Preview*

Elements:

- Before/after slider.
  - SVG zoom preview.
  - Background toggle.
  - Original overlay.
  - Path view.
  - Palette list.
  - Metrics.
-

## *Screen 5: Download*

Elements:

- Download SVG.
  - Download PNG.
  - Download PDF.
  - Download ZIP.
  - Re-run settings.
  - Save project.
  - Delete files.
  - API example snippet.
- 

## **26. Suggested Preset Details**

### *Logo/Icon*

Default settings:

- Upscale if short side is under 512 px.
- Preserve alpha.
- Max colors: 8-16.
- Shape fitting: on.
- Symmetry detection: on.
- Simplification: medium.
- Output: SVG, PNG, PDF.

### *Sticker/Vinyl/Cutter*

Default settings:

- Remove background.
- Reduce colors.
- Strong simplification.
- Avoid tiny shapes.
- Centerline option available.
- Output: SVG, DXF, PDF.

### *Line Art/Sketch*

Default settings:

- Monochrome or limited colors.

- Centerline mode available.
- Noise removal.
- Stroke preservation.
- Output: SVG, DXF, PDF.

### *Photo to Poster Vector*

Default settings:

- Warn user that output is artistic, not clean logo SVG.
- Palette reduction.
- Region simplification.
- Output: SVG, PNG, PDF.

### *AI Art Cleanup*

Default settings:

- Denoise.
- Upscale.
- Preserve style.
- Optional poster/vector export.
- Output: PNG, WebP, optional SVG.

### *Transparent PNG Upscale*

Default settings:

- Preserve alpha.
- Avoid matte halos.
- Edge-aware upscaling.
- Output: PNG, WebP.

---

## 27. Developer API Positioning

The API should be a major product differentiator because some competitors have limited or unavailable APIs.

Developer use cases:

- Print shops automating logo cleanup.
- Sticker businesses converting uploaded art.
- Design platforms offering “make SVG” buttons.
- AI image apps adding upscale/vector export.
- Marketplaces cleaning seller-uploaded images.

- SaaS tools generating brand kits.

API should include:

- Upload sessions.
  - Async jobs.
  - Status polling.
  - Webhooks.
  - Preview mode.
  - Production mode.
  - Re-download formats.
  - Retention controls.
  - Usage reporting.
  - Signed downloads.
- 

## 28. AI Bot Product Design

The AI bot should be simple and task-focused.

Example commands:

“Clean this logo and make SVG.”

“Upscale this 4x.”

“Make this transparent PNG sharper.”

“Vectorize this for vinyl cutting.”

“Turn this into SVG and PDF.”

“Use max 8 colors.”

“Remove the white background.”

Bot output should include:

- Diagnosis.
- Recommended mode.
- Preview.
- Download links.
- Optional advanced settings.

Example response:

“I detected a low-resolution logo with a white background. I removed the background, upscaled it 4x, and generated an 8-color SVG. The result is suitable for print and light editing. For vinyl cutting, I recommend increasing simplification.”

---

## 29. Mobile App Product Design

The mobile app should launch after the web app and API are proven.

Core features:

- Import from camera.
- Import from photo library.
- Import from Files.
- Share-sheet support.
- One-tap modes.
- Simple preview.
- Export/share.
- Push notification when processing is done.

Mobile modes:

- Clean Logo.
- Upscale Photo.
- Make SVG.
- Sticker/Cutter.

Mobile should not include complex controls at first. It should focus on speed and clarity.

Possible future mobile feature:

- AI logo generation.
- Logo redesign suggestions.
- Brand kit export.
- Social media asset generation.

---

## 30. Final Product Summary

Caliper Studio should be a professional image enhancement and vectorization platform.

It should accept a user-uploaded raster image and output:

- A sharper raster version.
- A clean editable vector version.
- Or both.

The strongest first version is a web app plus API. The AI bot should come next because it can use the same backend. Mobile should come later, once the processing pipeline and presets are reliable.

The core advantage should be the Assayer, which automatically determines the best workflow for the user's image.

Instead of forcing users to understand technical settings, the app should say things like:

"Low-resolution logo detected. Recommended: 4x upscale, 12-color SVG, preserve transparency."

This will make the product feel intelligent, professional, and easier to use than a simple converter.

The most commercially attractive first workflow is:

Low-resolution logo  
→ 4x conservative upscale  
→ 8-16 color vectorization  
→ Clean SVG preview  
→ SVG + PNG + PDF bundle

This fits small businesses, designers, print shops, sticker makers, and API customers.

---

## 31. Sources Consulted

Vectorizer.io home:  
<https://www.vectorizer.io/>

Vectorizer.io API:  
<https://www.vectorizer.io/api/>

Vectorizer.io pricing:  
<https://www.vectorizer.io/pricing/>

Vectorizer.AI home:  
<https://vectorizer.ai/>

Vectorizer.AI API:  
<https://vectorizer.ai/api>

Topaz Gigapixel:  
<https://www.topazlabs.com/topaz-gigapixel>

Topaz AI Image Upscaler:  
<https://www.topazlabs.com/tools/image-upscale>

Topaz Labs Image API quickstart:  
<https://developer.topazlabs.com/getting-started/quickstart>

Topaz Labs Image Enhance API:  
<https://developer.topazlabs.com/reference/api-endpoints/image/enhance>

Topaz Labs Image Tool API:  
<https://developer.topazlabs.com/reference/api-endpoints/image/tool>

Topaz Labs model selection:  
<https://developer.topazlabs.com/getting-started/model-selection>

Topaz Labs API pricing:  
<https://developer.topazlabs.com/getting-started/pricing>

Topaz Gigapixel docs:

<https://docs.topazlabs.com/topaz-gigapixel/enhancements/upscale>

Poe TopazLabs bot:

<https://poe.com/TopazLabs>

Poe TopazLabs demo:

<https://poe.com/pages/demos/upscale-your-images-with-topazlabs>

AI Logo Maker Google Play listing:

[Google Play App Link](#)

Real-ESRGAN GitHub:

<https://github.com/xinntao/Real-ESRGAN>

SwinIR paper:

<https://arxiv.org/abs/2108.10257>

Potrace technical documentation:

<https://potrace.sourceforge.net/potrace.pdf>

VTracer GitHub:

<https://github.com/visioncortex/vtracer>